

BlinkDB Storage Engine

1.0

Generated by Doxygen 1.9.8

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 BlinkDB Class Reference	5
3.1.1 Detailed Description	6
3.1.2 Constructor & Destructor Documentation	6
3.1.2.1 BlinkDB()	6
3.1.3 Member Function Documentation	7
3.1.3.1 compact()	7
3.1.3.2 del()	7
3.1.3.3 flush()	8
3.1.3.4 get()	9
3.1.3.5 search_sstables()	10
3.1.3.6 set()	11
3.1.4 Member Data Documentation	12
3.1.4.1 cache	12
3.1.4.2 capacity	12
3.1.4.3 size	12
3.1.4.4 sstables	12
3.1.4.5 store	12
3.2 BloomFilter Class Reference	13
3.2.1 Detailed Description	13
3.2.2 Constructor & Destructor Documentation	13
3.2.2.1 BloomFilter()	13
3.2.3 Member Function Documentation	14
3.2.3.1 add()	14
3.2.3.2 contains()	14
3.2.3.3 hash_i()	15
3.2.4 Member Data Documentation	15
3.2.4.1 bits	15
3.2.4.2 k_hashes	16
3.2.4.3 m_bits	16
3.3 BlinkDB::Entry Struct Reference	16
3.3.1 Detailed Description	16
3.3.2 Member Data Documentation	16
3.3.2.1 deleted	16
3.3.2.2 value	17
3.4 LRUCache< K, V > Class Template Reference	17
3.4.1 Detailed Description	17

3.4.2 Constructor & Destructor Documentation	18
3.4.2.1 LRUCache()	18
3.4.3 Member Function Documentation	18
3.4.3.1 get()	18
3.4.3.2 put()	19
3.4.3.3 remove()	19
3.4.4 Member Data Documentation	20
3.4.4.1 cacheList	20
3.4.4.2 cacheMap	20
3.4.4.3 capacity	20
4 File Documentation	21
4.1 blinkdb.cpp File Reference	21
4.1.1 Detailed Description	22
4.1.2 Function Documentation	22
4.1.2.1 binary_search_file()	22
4.1.2.2 build_index_for_sstable()	23
4.1.2.3 get_line_from_mapping()	24
4.1.2.4 main()	25
4.1.2.5 map_file()	25
4.1.3 Variable Documentation	26
4.1.3.1 sstable_filters	26
4.1.3.2 sstable_indices	26
4.1.3.3 sstable_mmaps	26
4.2 blinkdb.h File Reference	26
4.2.1 Detailed Description	27
4.3 blinkdb.h	28
4.4 lru_cache.h File Reference	28
4.4.1 Detailed Description	29
4.5 lru_cache.h	29
Index	31

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BlinkDB	A simple key-value storage engine	5
BloomFilter	A simple Bloom filter implementation	13
BlinkDB::Entry	Represents an entry in the in-memory memtable	16
LRUCache< K, V >	A template class for a Least Recently Used (LRU) cache	17

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

blinkdb.cpp	Implementation of the BlinkDB key-value storage engine and REPL	21
blinkdb.h	Header file for the BlinkDB key-value storage engine	26
lru_cache.h	Header file for the LRUCache template class	28

Chapter 3

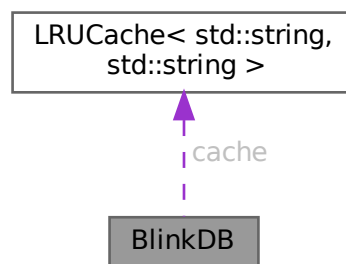
Class Documentation

3.1 BlinkDB Class Reference

A simple key-value storage engine.

```
#include <blinkdb.h>
```

Collaboration diagram for BlinkDB:



Classes

- struct [Entry](#)
Represents an entry in the in-memory memtable.

Public Member Functions

- [BlinkDB](#) (size_t [capacity](#)=3, size_t [cache_capacity](#)=10)
Constructs a [BlinkDB](#) instance.
- void [set](#) (const std::string &key, const std::string &value)
Sets the value for a given key.
- std::string [get](#) (const std::string &key)
Retrieves the value associated with a given key.
- bool [del](#) (const std::string &key)
Deletes a key-value pair.

Private Member Functions

- void [flush](#) ()
Flushes the in-memory memtable to an SSTable file.
- void [compact](#) ()
Compacts multiple SSTable files into a single SSTable.
- std::string [search_sstables](#) (const std::string &key) const
Searches for a key in all SSTable files.

Private Attributes

- std::unordered_map< std::string, [Entry](#) > [store](#)
In-memory memtable.
- size_t [capacity](#)
Maximum number of entries in the memtable.
- size_t [size](#)
Current number of entries in the memtable.
- std::vector< std::string > [sstables](#)
List of SSTable file names.
- [LRUCache](#)< std::string, std::string > [cache](#)
LRU cache used to speed up GET operations.

3.1.1 Detailed Description

A simple key-value storage engine.

The [BlinkDB](#) class provides functionality for setting, getting, and deleting key-value pairs. It uses an in-memory memtable and persistent SSTable files. It also leverages an LRU cache for fast retrieval of frequently accessed keys.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 BlinkDB()

```
BlinkDB::BlinkDB (
    size_t capacity = 3,
    size_t cache_capacity = 10 ) [inline], [explicit]
```

Constructs a [BlinkDB](#) instance.

The constructor initializes the memtable capacity and the LRU cache capacity.

Parameters

<i>capacity</i>	Maximum number of entries for the memtable. Default is 3.
<i>cache_capacity</i>	Capacity for the LRU cache. Default is 10.

3.1.3 Member Function Documentation

3.1.3.1 compact()

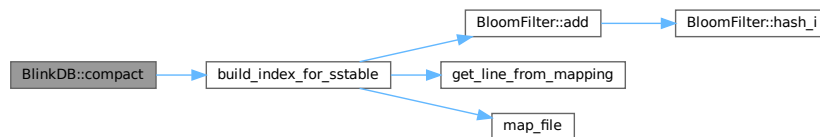
```
void BlinkDB::compact ( ) [private]
```

Compacts multiple SSTable files into a single SSTable.

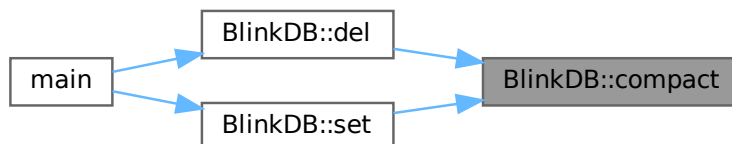
Compacts SSTable files by merging them into a single file.

This function merges SSTable files and removes obsolete entries.

This function merges all SSTable files into one, removes tombstones, and rebuilds the in-memory indices, Bloom filters, and memory maps. Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.2 del()

```
bool BlinkDB::del (
    const std::string & key )
```

Deletes a key-value pair.

Deletes the key-value pair for a given key.

The key is marked as deleted in the memtable.

Parameters

<i>key</i>	The key to delete.
------------	--------------------

Returns

true if the key existed and was marked for deletion, false otherwise.

The deletion is performed in the in-memory store and, if necessary, a tombstone is added.

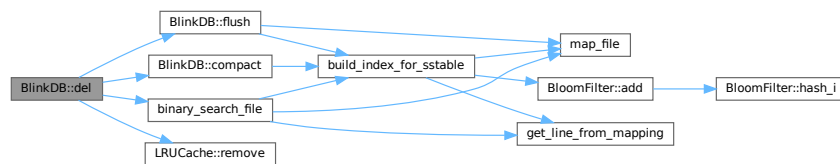
Parameters

<i>key</i>	The key to delete.
------------	--------------------

Returns

true if the key existed and was marked for deletion, false otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.1.3.3 flush()**

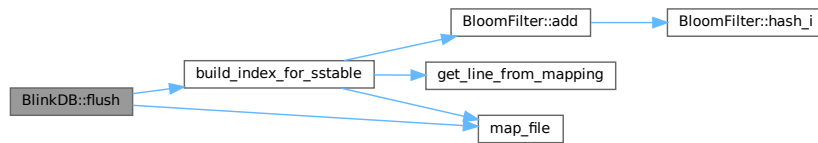
```
void BlinkDB::flush ( ) [private]
```

Flushes the in-memory memtable to an SSTable file.

Flushes the in-memory store to disk as a new SSTable file.

This function writes the current memtable to disk as an SSTable file.

The function writes all key-value pairs to a file and updates the in-memory index, Bloom filter, and memory mapping for the new SSTable. Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.4 get()

```
std::string BlinkDB::get (
    const std::string & key )
```

Retrieves the value associated with a given key.

The function checks the memtable, cache, and SSTable files in order.

Parameters

<i>key</i>	The key to retrieve.
------------	----------------------

Returns

The value associated with the key, or "NULL" if not found.

The function first checks the LRU cache, then the in-memory store, and finally the SSTable files.

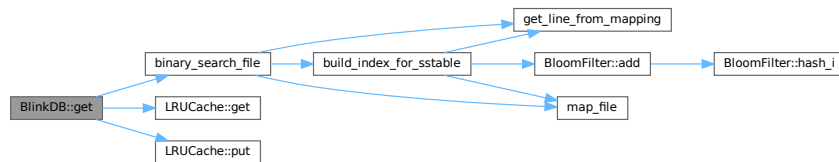
Parameters

<i>key</i>	The key to retrieve.
------------	----------------------

Returns

The associated value, or "NULL" if not found.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.1.3.5 search_sstables()**

```
std::string BlinkDB::search_sstables (
    const std::string & key ) const [private]
```

Searches for a key in all SSTable files.

The search is performed on the SSTable files if the key is not found in the memtable.

Parameters

<i>key</i>	The key to search for.
------------	------------------------

Returns

The value associated with the key, or "NULL" if not found.

The search is performed from the newest to the oldest SSTable.

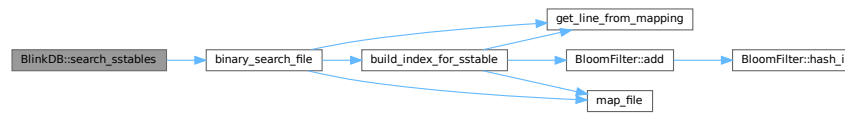
Parameters

<i>key</i>	The key to search for.
------------	------------------------

Returns

The associated value if found, otherwise "NULL".

Here is the call graph for this function:

**3.1.3.6 set()**

```
void BlinkDB::set (
    const std::string & key,
    const std::string & value )
```

Sets the value for a given key.

Sets the value for a given key in the storage engine.

If the key already exists, its value is updated.

Parameters

<i>key</i>	The key to set.
<i>value</i>	The value to associate with the key.

This method updates the in-memory store and the LRU cache.

Parameters

<i>key</i>	The key to set.
<i>value</i>	The value to associate with the key.

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.4 Member Data Documentation

3.1.4.1 cache

```
LRUCache<std::string, std::string> BlinkDB::cache [private]
```

LRU cache used to speed up GET operations.

3.1.4.2 capacity

```
size_t BlinkDB::capacity [private]
```

Maximum number of entries in the memtable.

3.1.4.3 size

```
size_t BlinkDB::size [private]
```

Current number of entries in the memtable.

3.1.4.4 sstables

```
std::vector<std::string> BlinkDB::sstables [private]
```

List of SSTable file names.

3.1.4.5 store

```
std::unordered_map<std::string, Entry> BlinkDB::store [private]
```

In-memory memtable.

The documentation for this class was generated from the following files:

- [blinkdb.h](#)
- [blinkdb.cpp](#)

3.2 BloomFilter Class Reference

A simple Bloom filter implementation.

Public Member Functions

- [BloomFilter](#) (size_t m, size_t k)
Construct a new [BloomFilter](#) object.
- void [add](#) (const std::string &s)
Adds a string element to the Bloom filter.
- bool [contains](#) (const std::string &s) const
Checks if a string element might be in the Bloom filter.

Private Member Functions

- size_t [hash_i](#) (const std::string &s, size_t i) const
Hash function combining std::hash with a seed.

Private Attributes

- size_t [m_bits](#)
Number of bits in the filter.
- size_t [k_hashes](#)
Number of hash functions.
- std::vector< bool > [bits](#)
Bit vector representing the Bloom filter.

3.2.1 Detailed Description

A simple Bloom filter implementation.

This class provides methods to add an element to the filter and check if an element might be present in the filter.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 BloomFilter()

```
BloomFilter::BloomFilter (
    size_t m,
    size_t k ) [inline]
```

Construct a new [BloomFilter](#) object.

Parameters

<i>m</i>	Number of bits in the filter.
<i>k</i>	Number of hash functions to use.

3.2.3 Member Function Documentation

3.2.3.1 add()

```
void BloomFilter::add (
    const std::string & s ) [inline]
```

Adds a string element to the Bloom filter.

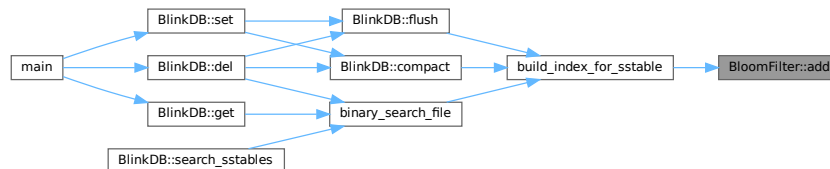
Parameters

s	The string element to add.
---	----------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



3.2.3.2 contains()

```
bool BloomFilter::contains (
    const std::string & s ) const [inline]
```

Checks if a string element might be in the Bloom filter.

Parameters

s	The string element to check.
---	------------------------------

Returns

true if the element might be in the filter, false otherwise.

Here is the call graph for this function:

**3.2.3.3 hash_i()**

```

size_t BloomFilter::hash_i (
    const std::string & s,
    size_t i ) const [inline], [private]
  
```

Hash function combining `std::hash` with a seed.

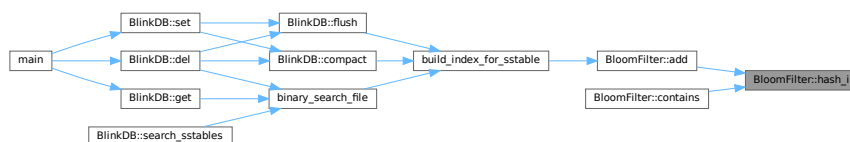
Parameters

<i>s</i>	The input string to hash.
<i>i</i>	The hash function seed.

Returns

The computed hash value.

Here is the caller graph for this function:

**3.2.4 Member Data Documentation****3.2.4.1 bits**

```
std::vector<bool> BloomFilter::bits [private]
```

Bit vector representing the Bloom filter.

3.2.4.2 k_hashes

```
size_t BloomFilter::k_hashes [private]
```

Number of hash functions.

3.2.4.3 m_bits

```
size_t BloomFilter::m_bits [private]
```

Number of bits in the filter.

The documentation for this class was generated from the following file:

- [blinkdb.cpp](#)

3.3 BlinkDB::Entry Struct Reference

Represents an entry in the in-memory memtable.

Public Attributes

- std::string [value](#)
The value associated with the key.
- bool [deleted](#) = false
Flag indicating if the entry is marked as deleted.

3.3.1 Detailed Description

Represents an entry in the in-memory memtable.

Each entry holds a string value and a flag indicating whether it has been deleted.

3.3.2 Member Data Documentation

3.3.2.1 deleted

```
bool BlinkDB::Entry::deleted = false
```

Flag indicating if the entry is marked as deleted.

3.3.2.2 value

```
std::string BlinkDB::Entry::value
```

The value associated with the key.

The documentation for this struct was generated from the following file:

- [blinkdb.h](#)

3.4 LRUCache< K, V > Class Template Reference

A template class for a Least Recently Used (LRU) cache.

```
#include <lru_cache.h>
```

Public Member Functions

- [LRUCache](#) (size_t capacity)
Constructs an [LRUCache](#) object with the specified capacity.
- bool [get](#) (const K &key, V &value)
Retrieves the value associated with a given key.
- void [put](#) (const K &key, const V &value)
Inserts or updates a key-value pair in the cache.
- void [remove](#) (const K &key)
Removes a key and its associated value from the cache.

Private Attributes

- size_t [capacity](#)
Maximum capacity of the cache.
- std::list< std::pair< K, V > > [cacheList](#)
List storing key-value pairs; front is most recently used.
- std::unordered_map< K, typename std::list< std::pair< K, V > >::iterator > [cacheMap](#)
Map for fast lookup of cache items.

3.4.1 Detailed Description

```
template<typename K, typename V>
class LRUCache< K, V >
```

A template class for a Least Recently Used (LRU) cache.

This class implements a cache that evicts the least recently used item when the capacity is reached. It supports basic operations like getting, putting, and removing items.

Template Parameters

<i>K</i>	Type of the key.
<i>V</i>	Type of the value.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 LRUCache()

```
template<typename K , typename V >
LRUCache< K, V >::LRUCache (
    size_t capacity ) [inline], [explicit]
```

Constructs an [LRUCache](#) object with the specified capacity.

Parameters

<i>capacity</i>	The maximum number of items the cache can hold.
-----------------	---

3.4.3 Member Function Documentation

3.4.3.1 get()

```
template<typename K , typename V >
bool LRUCache< K, V >::get (
    const K & key,
    V & value ) [inline]
```

Retrieves the value associated with a given key.

If the key is found, the value is stored in the provided reference and the item is moved to the front of the cache (marking it as most recently used).

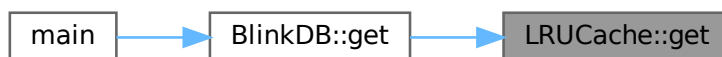
Parameters

<i>key</i>	The key to retrieve.
<i>value</i>	Output parameter to store the retrieved value.

Returns

true if the key is found, false otherwise.

Here is the caller graph for this function:

**3.4.3.2 put()**

```

template<typename K , typename V >
void LRUCache< K, V >::put (
    const K & key,
    const V & value ) [inline]
  
```

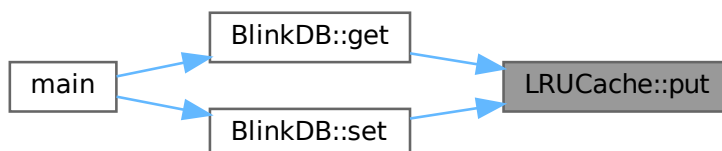
Inserts or updates a key-value pair in the cache.

If the key already exists, its value is updated and the item is moved to the front. If the cache exceeds its capacity, the least recently used item is removed.

Parameters

<i>key</i>	The key to insert or update.
<i>value</i>	The value associated with the key.

Here is the caller graph for this function:

**3.4.3.3 remove()**

```

template<typename K , typename V >
void LRUCache< K, V >::remove (
    const K & key ) [inline]
  
```

Removes a key and its associated value from the cache.

If the key exists, it is removed from both the internal list and map.

Parameters

<i>key</i>	The key to remove.
------------	--------------------

Here is the caller graph for this function:



3.4.4 Member Data Documentation

3.4.4.1 cacheList

```
template<typename K , typename V >
std::list<std::pair<K, V> > LRUCache< K, V >::cacheList [private]
```

List storing key-value pairs; front is most recently used.

3.4.4.2 cacheMap

```
template<typename K , typename V >
std::unordered_map<K, typename std::list<std::pair<K, V> >::iterator> LRUCache< K, V >↔
::cacheMap [private]
```

Map for fast lookup of cache items.

3.4.4.3 capacity

```
template<typename K , typename V >
size_t LRUCache< K, V >::capacity [private]
```

Maximum capacity of the cache.

The documentation for this class was generated from the following file:

- [lru_cache.h](#)

Chapter 4

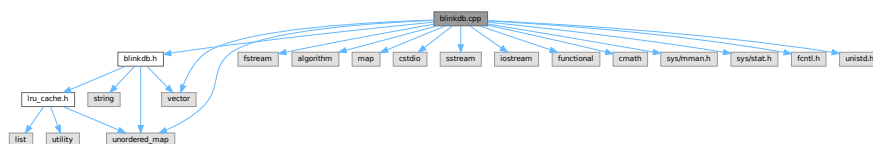
File Documentation

4.1 blinkdb.cpp File Reference

Implementation of the [BlinkDB](#) key-value storage engine and REPL.

```
#include "blinkdb.h"
#include <fstream>
#include <algorithm>
#include <vector>
#include <map>
#include <cstdio>
#include <sstream>
#include <iostream>
#include <unordered_map>
#include <functional>
#include <cmath>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
```

Include dependency graph for blinkdb.cpp:



Classes

- class [BloomFilter](#)
A simple Bloom filter implementation.

Functions

- static const char * [map_file](#) (const std::string &filename, size_t &filesize)
Memory-map a file and return a pointer to its content along with its size.
- static std::string [get_line_from_mapping](#) (const char *data, size_t filesize, size_t offset)
Retrieve a full line from a memory-mapped file starting at a given offset.
- static std::vector< std::pair< std::string, std::streampos > > [build_index_for_sstable](#) (const std::string &filename)
Build an in-memory index for an SSTable file and create its Bloom filter.
- static std::string [binary_search_file](#) (const std::string &filename, const std::string &key)
Perform a binary search on the in-memory index for a given SSTable file.
- int [main](#) ()
Main function providing a REPL interface for the [BlinkDB](#) storage engine.

Variables

- static std::unordered_map< std::string, std::vector< std::pair< std::string, std::streampos > > > [sstable_indices](#)
- static std::unordered_map< std::string, [BloomFilter](#) > [sstable_filters](#)
- static std::unordered_map< std::string, std::pair< const char *, size_t > > [sstable_mmaps](#)

4.1.1 Detailed Description

Implementation of the [BlinkDB](#) key-value storage engine and REPL.

This file implements the core functionality of the [BlinkDB](#) storage engine, including the Bloom filter, in-memory indices for SSTables, and helper functions for memory mapping. It also contains the main REPL for interacting with the storage engine.

4.1.2 Function Documentation

4.1.2.1 [binary_search_file\(\)](#)

```
static std::string binary_search_file (
    const std::string & filename,
    const std::string & key ) [static]
```

Perform a binary search on the in-memory index for a given SSTable file.

First, the Bloom filter is checked to quickly rule out a missing key. If the key might exist, a binary search is performed on the index and the corresponding record is retrieved.

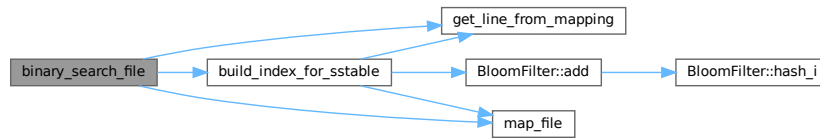
Parameters

<i>filename</i>	The name of the SSTable file.
<i>key</i>	The key to search for.

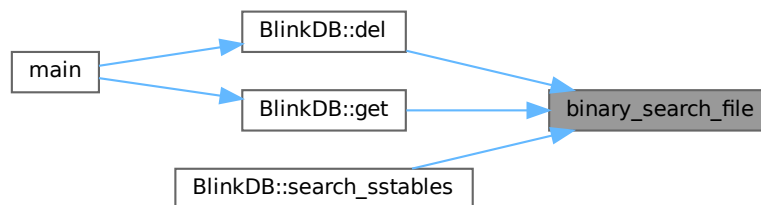
Returns

The value associated with the key, or "NULL" if not found.

Here is the call graph for this function:



Here is the caller graph for this function:

**4.1.2.2 build_index_for_sstable()**

```
static std::vector< std::pair< std::string, std::streampos > > build_index_for_sstable (
    const std::string & filename ) [static]
```

Build an in-memory index for an SSTable file and create its Bloom filter.

The function reads each record (line) from the SSTable, extracts the key, and records its file offset. It then builds a Bloom filter based on the keys.

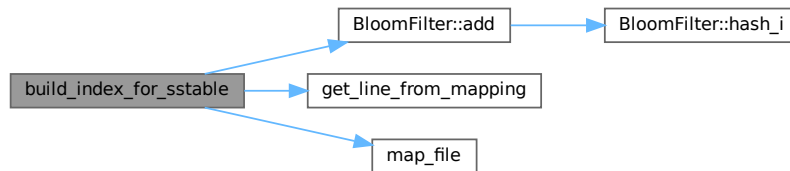
Parameters

<i>filename</i>	The name of the SSTable file.
-----------------	-------------------------------

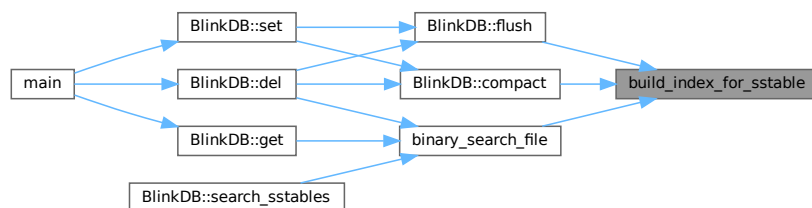
Returns

A vector of (key, file offset) pairs sorted by key.

Here is the call graph for this function:



Here is the caller graph for this function:

**4.1.2.3 get_line_from_mapping()**

```

static std::string get_line_from_mapping (
    const char * data,
    size_t filesize,
    size_t offset ) [static]
  
```

Retrieve a full line from a memory-mapped file starting at a given offset.

The returned line does not include the newline character.

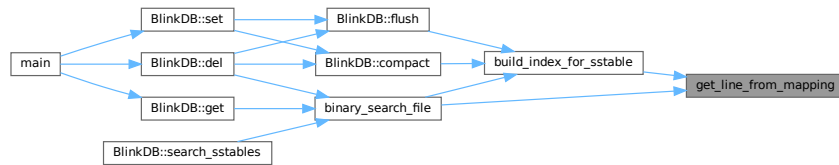
Parameters

<i>data</i>	Pointer to the mapped file data.
<i>filesize</i>	Size of the file.
<i>offset</i>	Offset in the file from which to start reading.

Returns

The line read from the file.

Here is the caller graph for this function:



4.1.2.4 main()

```
int main ( )
```

Main function providing a REPL interface for the [BlinkDB](#) storage engine.

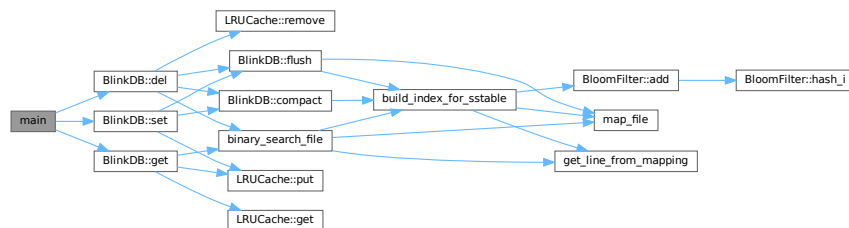
The REPL supports the following commands:

- SET <key> "<value>"
- GET <key>
- DEL <key>
- EXIT

Returns

int Exit status code.

Here is the call graph for this function:



4.1.2.5 map_file()

```
static const char * map_file (
    const std::string & filename,
    size_t & filesize ) [static]
```

Memory-map a file and return a pointer to its content along with its size.

If the file has already been mapped, the cached mapping is returned.

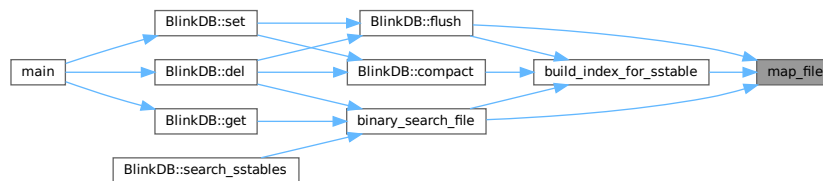
Parameters

<i>filename</i>	The name of the file to map.
<i>filesize</i>	Output parameter for the size of the file.

Returns

A pointer to the mapped file data, or nullptr on error.

Here is the caller graph for this function:



4.1.3 Variable Documentation

4.1.3.1 sstable_filters

```
std::unordered_map<std::string, BloomFilter> sstable_filters [static]
```

4.1.3.2 sstable_indices

```
std::unordered_map<std::string, std::vector<std::pair<std::string, std::streampos> > > sstable←
_indices [static]
```

4.1.3.3 sstable_mmaps

```
std::unordered_map<std::string, std::pair<const char*, size_t> > sstable_mmaps [static]
```

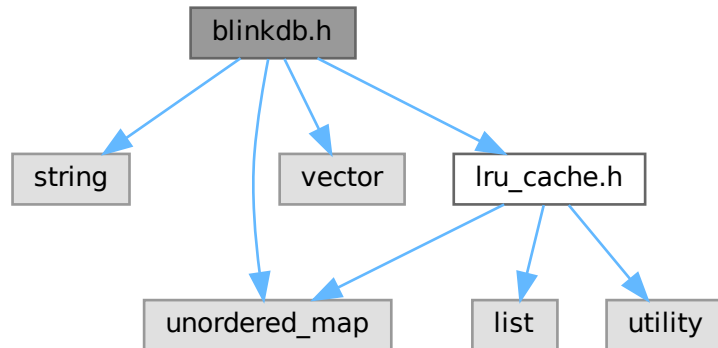
4.2 blinkdb.h File Reference

Header file for the [BlinkDB](#) key-value storage engine.

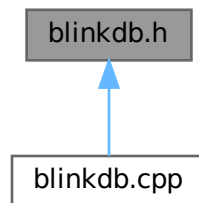
```
#include <string>
#include <unordered_map>
#include <vector>
```

```
#include "lru_cache.h"
```

Include dependency graph for blinkdb.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [BlinkDB](#)
A simple key-value storage engine.
- struct [BlinkDB::Entry](#)
Represents an entry in the in-memory memtable.

4.2.1 Detailed Description

Header file for the [BlinkDB](#) key-value storage engine.

This file declares the [BlinkDB](#) class which implements a simple key-value storage engine with in-memory memtable and SSTable persistence, along with an LRU cache for fast GET operations.

4.3 blinkdb.h

[Go to the documentation of this file.](#)

```

00001
00009 #pragma once
00010
00011 #include <string>
00012 #include <unordered_map>
00013 #include <vector>
00014 #include "lru_cache.h" // include the LRU cache header
00015
00024 class BlinkDB
00025 {
00026 private:
00033     struct Entry
00034     {
00035         std::string value;
00036         bool deleted = false;
00037     };
00038
00039     std::unordered_map<std::string, Entry> store;
00040     size_t capacity;
00041     size_t size;
00042     std::vector<std::string> sstables;
00043
00049     void flush();
00050
00056     void compact();
00057
00066     std::string search_sstables(const std::string &key) const;
00067
00071     LRUCache<std::string, std::string> cache;
00072
00073 public:
00082     explicit BlinkDB(size_t capacity = 3, size_t cache_capacity = 10)
00083         : capacity(capacity), size(0), cache(cache_capacity) {}
00084
00093     void set(const std::string &key, const std::string &value);
00094
00103     std::string get(const std::string &key);
00104
00113     bool del(const std::string &key);
00114 };
00115

```

4.4 lru_cache.h File Reference

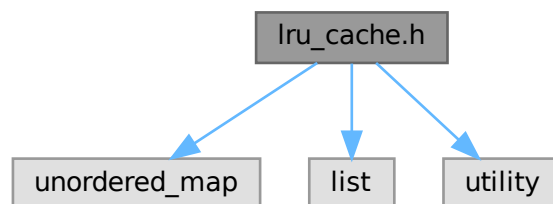
Header file for the [LRUCache](#) template class.

```

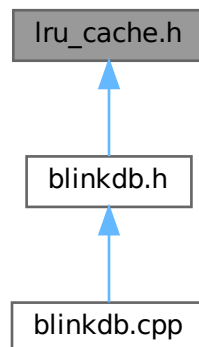
#include <unordered_map>
#include <list>
#include <utility>

```

Include dependency graph for lru_cache.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `LRUCache< K, V >`
A template class for a Least Recently Used (LRU) cache.

4.4.1 Detailed Description

Header file for the `LRUCache` template class.

This file declares the `LRUCache` template class, which provides a simple implementation of a least-recently-used (LRU) cache.

4.5 lru_cache.h

[Go to the documentation of this file.](#)

```

00001 #ifndef LRUCACHE_H
00002 #define LRUCACHE_H
00003
00012 #include <unordered_map>
00013 #include <list>
00014 #include <utility>
00015
00026 template<typename K, typename V>
00027 class LRUCache {
00028 public:
00033     explicit LRUCache(size_t capacity) : capacity(capacity) {}
00034
00045     bool get(const K &key, V &value) {
00046         auto it = cacheMap.find(key);
00047         if (it == cacheMap.end())
00048             return false;
00049         // Move accessed item to the front (most recently used)
00050         cacheList.splice(cacheList.begin(), cacheList, it->second);
00051         value = it->second->second;
00052         return true;
00053     }
00054
00064     void put(const K &key, const V &value) {
00065         auto it = cacheMap.find(key);
  
```

```
00066         if (it != cacheMap.end()) {
00067             // Update value and move to front
00068             it->second->second = value;
00069             cacheList.splice(cacheList.begin(), cacheList, it->second);
00070             return;
00071         }
00072         // Check capacity, remove least recently used if needed
00073         if (cacheList.size() >= capacity) {
00074             auto last = cacheList.back();
00075             cacheMap.erase(last.first);
00076             cacheList.pop_back();
00077         }
00078         cacheList.emplace_front(key, value);
00079         cacheMap[key] = cacheList.begin();
00080     }
00081
00082     void remove(const K &key) {
00083         auto it = cacheMap.find(key);
00084         if (it != cacheMap.end()) {
00085             cacheList.erase(it->second);
00086             cacheMap.erase(it);
00087         }
00088     }
00089
00090 private:
00091     size_t capacity;
00092     std::list<std::pair<K, V> > cacheList;
00093     std::unordered_map<K, typename std::list<std::pair<K, V>::iterator> > cacheMap;
00094 };
00095
00096 #endif // LRUCACHE_H
```

Index

- add
 - BloomFilter, [14](#)
- binary_search_file
 - blinkdb.cpp, [22](#)
- bits
 - BloomFilter, [15](#)
- BlinkDB, [5](#)
 - BlinkDB, [6](#)
 - cache, [12](#)
 - capacity, [12](#)
 - compact, [7](#)
 - del, [7](#)
 - flush, [8](#)
 - get, [9](#)
 - search_sstables, [10](#)
 - set, [11](#)
 - size, [12](#)
 - sstables, [12](#)
 - store, [12](#)
- blinkdb.cpp, [21](#)
 - binary_search_file, [22](#)
 - build_index_for_sstable, [23](#)
 - get_line_from_mapping, [24](#)
 - main, [25](#)
 - map_file, [25](#)
 - sstable_filters, [26](#)
 - sstable_indices, [26](#)
 - sstable_mmaps, [26](#)
- blinkdb.h, [26](#)
- BlinkDB::Entry, [16](#)
 - deleted, [16](#)
 - value, [16](#)
- BloomFilter, [13](#)
 - add, [14](#)
 - bits, [15](#)
 - BloomFilter, [13](#)
 - contains, [14](#)
 - hash_i, [15](#)
 - k_hashes, [15](#)
 - m_bits, [16](#)
- build_index_for_sstable
 - blinkdb.cpp, [23](#)
- cache
 - BlinkDB, [12](#)
- cacheList
 - LRUCache< K, V >, [20](#)
- cacheMap
 - LRUCache< K, V >, [20](#)
- capacity
 - BlinkDB, [12](#)
 - LRUCache< K, V >, [20](#)
- compact
 - BlinkDB, [7](#)
- contains
 - BloomFilter, [14](#)
- del
 - BlinkDB, [7](#)
- deleted
 - BlinkDB::Entry, [16](#)
- flush
 - BlinkDB, [8](#)
- get
 - BlinkDB, [9](#)
 - LRUCache< K, V >, [18](#)
- get_line_from_mapping
 - blinkdb.cpp, [24](#)
- hash_i
 - BloomFilter, [15](#)
- k_hashes
 - BloomFilter, [15](#)
- lru_cache.h, [28](#)
- LRUCache
 - LRUCache< K, V >, [18](#)
- LRUCache< K, V >, [17](#)
 - cacheList, [20](#)
 - cacheMap, [20](#)
 - capacity, [20](#)
 - get, [18](#)
 - LRUCache, [18](#)
 - put, [19](#)
 - remove, [19](#)
- m_bits
 - BloomFilter, [16](#)
- main
 - blinkdb.cpp, [25](#)
- map_file
 - blinkdb.cpp, [25](#)
- put
 - LRUCache< K, V >, [19](#)
- remove

- LRUCache< K, V >, [19](#)
- search_sstables
 - BlinkDB, [10](#)
- set
 - BlinkDB, [11](#)
- size
 - BlinkDB, [12](#)
- sstable_filters
 - blinkdb.cpp, [26](#)
- sstable_indices
 - blinkdb.cpp, [26](#)
- sstable_mmaps
 - blinkdb.cpp, [26](#)
- sstables
 - BlinkDB, [12](#)
- store
 - BlinkDB, [12](#)
- value
 - BlinkDB::Entry, [16](#)